Progetto di Reti Logiche AA 2022–2023

Edoardo Egidio

Contents

1 Introduzione **2**
- 1.1 Scopo del progetto . . . 2
- 1.2 Interfaccia . . . 2
- 1.3 Specifica . . . 3

2 Architettura **3**
- 2.1 Overwiew . . . 3
- 2.2 FSM . . . 4
  - 2.2.1 Stati dell'automa . . . 5
- 2.3 Registri . . . 5

3 Risultati sperimentali **6**
- 3.1 Simulazione . . . 6
  - 3.1.1 Test 1 . . . 6
  - 3.1.2 Test 2 . . . 7
- 3.2 Sintesi . . . 9

4 Conclusioni **9**

1 Introduzione

1.1 Scopo del progetto

Lo scopo del progetto è di implementare un componente HW descritto in VHDL che sia in grado di deserializzare un segnale in ingresso di lunghezza variabile. L'ingresso fornisce un indirizzo di memoria ed indica uno dei quattro canali di uscita su cui instradare il dato letto a tale indirizzo.

1.2 Interfaccia

Il modulo da implementare ha tale intefaccia:

```
entity project_reti_logiche is
    port(
        i_clk   :  in std_logic;
        i_rst   :  in std_logic;
        i_start :  in std_logic;
        i_w     :  in std_logic;

        o_z0    :  out std_logic_vector(7 downto 0);
        o_z1    :  out std_logic_vector(7 downto 0);
        o_z2    :  out std_logic_vector(7 downto 0);
        o_z3    :  out std_logic_vector(7 downto 0);
        o_done  :  out std_logic;

        o_mem_addr  : out std_logic_vector(15 downto 0);
        i_mem_data  : in std_logic_vector(7 downto 0);
        o_mem_we    : out std_logic;
        o_mem_en    : out std_logic;
    );
end project_reti_logiche;
```

Nello specifico:

- `i_clk` è il segnale di clock ($\geq$10 MHz);
- `i_rst` è il segnale di reset asincrono del modulo;
- `i_start` è il sengale che rende valida la sequenza di input;
- `i_w` è l'ingresso seriale da processare;
- `o_z0, o_z1, o_z2, o_z3` sono i 4 canali di uscita;

- `o_done` è il segnale che notifica l'avvenuta lettura;
- `o_mem_addr` è il vettore contenete l'indirizzo di memoria deserializzato;
- `i_mem_data` è il vettore contenete il dato letto da memoria
- `o_mem_en` è il sengale di ENABLE per poter comunicare con la memoria sia in lettura che in scrittura;
- `o_mem_we` è il sengale di WRITE ENABLE per poter scrivere in memoria, il modulo lo terrà a 0 dato che non deve scriverci;

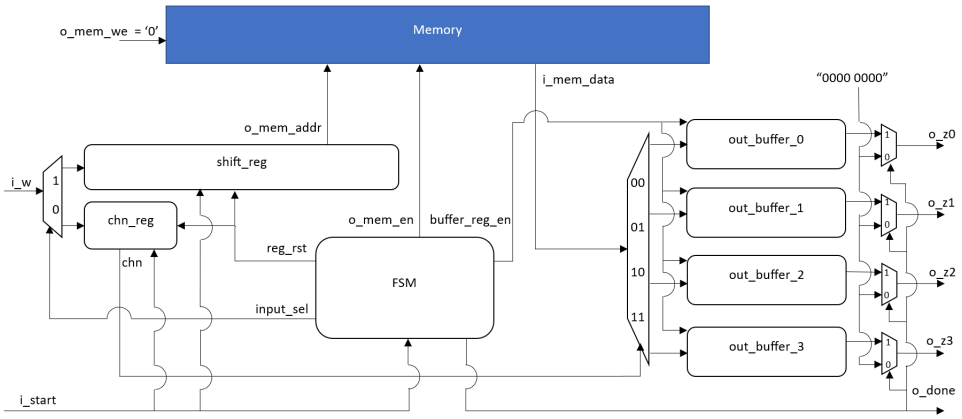
1.3 Specifica

Il modulo da sintetizzare deve leggere una sequenza di bit su `i_w` (da leggere sul fronte di salita del clock) solamente quando `i_start` è alto. `i_start` rimane alto per $2 \leq N \leq 18$ cicli di clock, e rimane basso fintanto che `o_done` non è tornato basso. Qunado `i_start` torna a 0 la sequenza di ingresso deve essere scomposta: i primi 2 bit indicano il canale di uscita da utilizzare, gli altri $N - 2$ devono essere estesi a sinistra con zeri fino ad arrivare a 16 bit. La parola ottenuta costituisce un indirizzo di memeoria a cui reperire un dato da 8 bit, da propagare poi sull'uscita prima selezionata, le altre uscite dovranno mostrare invece l'ultimo messaggio trasmesso. Tuttavia le uscite dovranno mostrare il valore solo mentre `o_done` è alto e solamente per un ciclo di clock, altrimenti saranno poste a "00".

2 Architettura

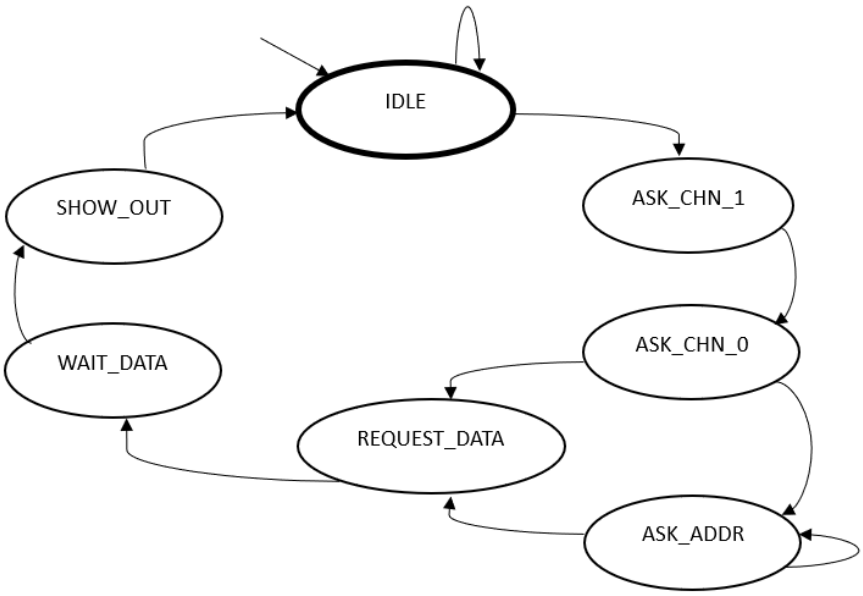
2.1 Overwiew

La entity sopra riportata è implementata tramite un'unica `architecture` di tipo `Behavioral` (con qualche elemento in `Dataflow`). L'idea è di utilizzare 2 registri a scorrimento (serie-parallelo) per deserializzare l'input, distinguendo i 2 bit del canale dagli $N - 2$ bit dell'indirzzo ($2 \leq N \leq 18$) tramite un demultiplexer. I dati letti da memoria vengono instradati con un secondo demultiplexer verso 1 dei 4 registri parallelo-parallelo, per poi essere connessi alle uscite quando `o_done` è alto. Il tutto viene controllato da una macchina di Mealy che cambia stato sul fronte di salita di `i_clock` e che ha come input `i_start`. Quato detto è schematizzato nella figura sottostante, per chiarezza sono stati omessi `i_clk`, `i_rst` (comuni a tutti gli elementi).



2.2 FSM

L'automa a stati finiti è implementato con due process: il primo si occupa di resettare la macchina quando `i_rst` è alto e di mutare lo stato quando `i_clk` sale, il secondo costituisce la funzione di stato prossimo che in base allo stato corrente e al valore di `i_start` definisce lo stato successivo. Di seguito è riportato il diagramma degli stati dell'automa.



2.2.1 Stati dell'automa

IDLE Stato iniziale in cui l'automa si trova dopo che è stato dato il reset o dopo che `o_done` è tornato a 0, se viene rilevato `i_start` alto allora si passa in ASK_CHN_1, inoltre `input_sel` viene posto a 0 per connettere `i_w` al registro `chn`.

ASK_CHN_1 Stato in cui viene letto il MSB bit del canale (`chn(1)`), lo stato successivo sarà ASK_CHN_0.

ASK_CHN_0 Stato in cui viene letto il LSB del canale (`chn(0)`), se `i_start` è ancora alto si passerà in ASK_ADDR e si porrà `input_sel` a 1, altrimenti si passerà in REQUEST_DATA.

ASK_ADDR Stato in cui viengono letti i bit dell'indirizzo, vi si rimane fino a che `i_start` è alto, mentre quando `i_start` è basso si passa in REQUEST_DATA.

REQUEST_DATA Stato in cui viene richiesto il dato, pertanto viene alzato `o_mem_en` per poter leggere da memoria, si passerà in WAIT_DATA.

WAIT_DATA Stato in cui si legge da memoria e si scrive nei registri di buffer alzando `buffer_reg_en`, si passerà in SHOW_OUT.

SHOW_OUT Stato in cui viene alzato `o_done` e vengono collegati i registri di buffer alle uscite, viene inoltre alzato `reg_rst` per azzerare i registri a scorrimento, lo stato successivo sarà IDLE.

2.3 Registri

I registri a scorrimento `chn` e `shift_reg`, rispettivamente da 2 e 16 bit, sono implementati ciascuno con un process che li azzera quando `i_rst` o `reg_rst` sono alti, altrimenti legge il valore di `i_w` sul fronte di salita del clock, coerentemente con il valore di `input_sel` (demultiplexer). Per quanto riguarda invece i registri di buffer (`out_buffer_0..3`), sono realizzati tramite un unico process che li azzera quando `i_rst` è alto, altrimenti viene scritto `i_mem_data` nel registro selezionato dal valore di `chn` (è necessario `buffer_reg_en` alto). Non sono stati implementati esplicitamente dei multiplexer per le uscite in quanto la funzione di stato prossimo dell'automa mette a "00" le uscite ad inizio del process ed essere vengono collegate ai registri di buffer solamente se lo stato corrente è SHOW_OUT, quindi quando `o_done` è alto.

3 Risultati sperimentali

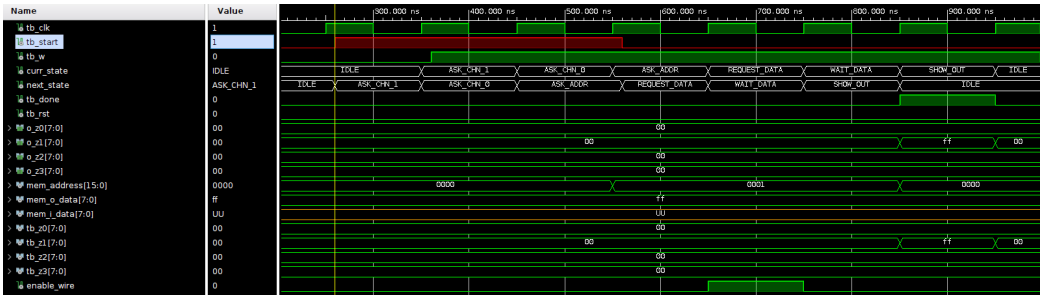
3.1 Simulazione

Oltre ad eseguire test bench di esempio sono stati scritti altri test per verificare il corretto comportamento del modulo. Data le limitazioni della specifica i sengali di input che potrebbero compromettere il funzionamento del componente sono `i_start` e `i_rst`: pertanto i test bench si focalizzano su questi.

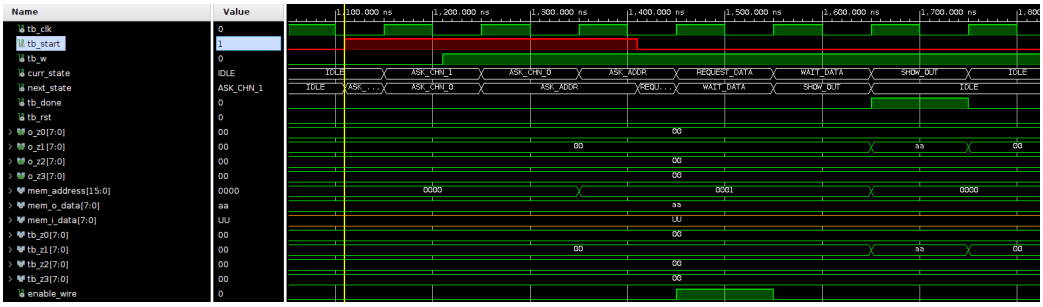
3.1.1 Test 1

Questo test bench verifica che il modulo operi correttamente con il segnale `i_start` che sale in diverse fasi del ciclo di clock, `i_start` viene tenuto alto per 500 ns e il dato letto da memoria è "FF" o "AA".
Nello specifico:

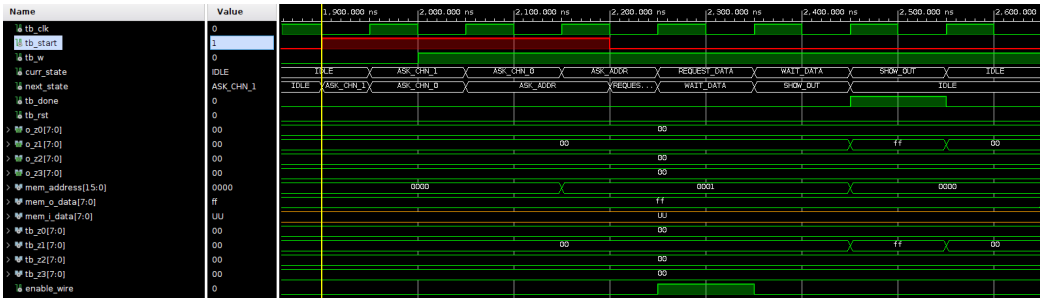
`i_start` sale poco dopo fronte di salita di `i_clk`



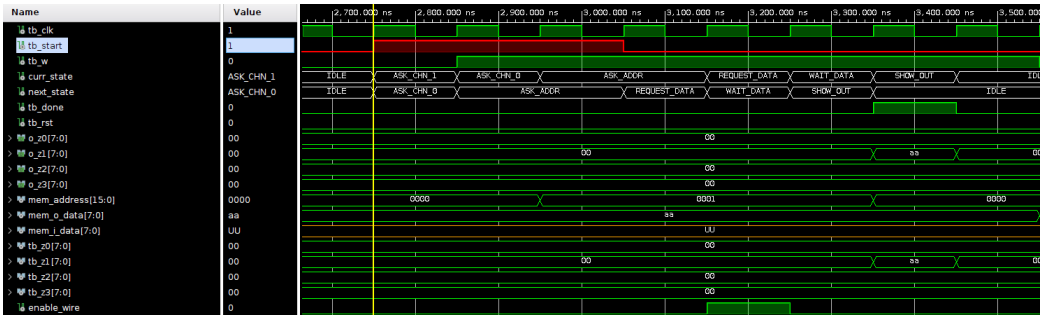
`i_start` sale poco prima il fronte di salita di `i_clk`



`i_start` sale durante fronte di discesa di `i_clk`



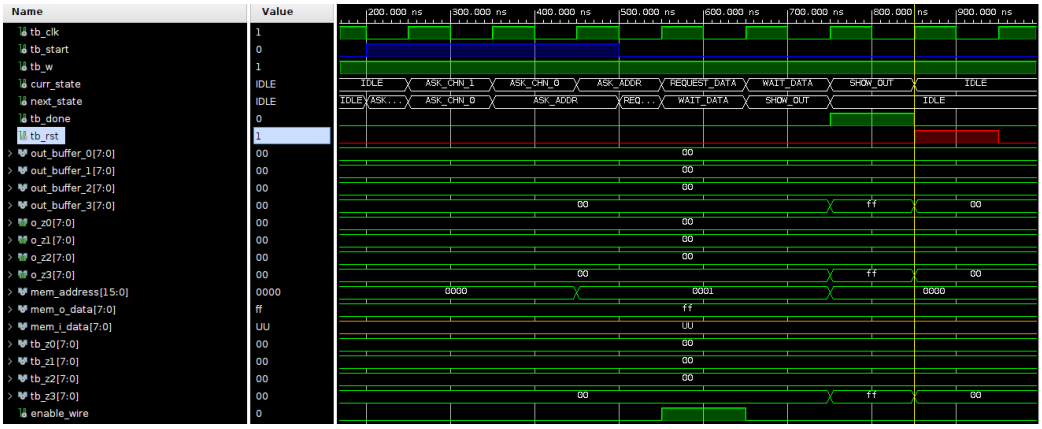
`i_start` sale durante il fronte di salita di `i_clk`



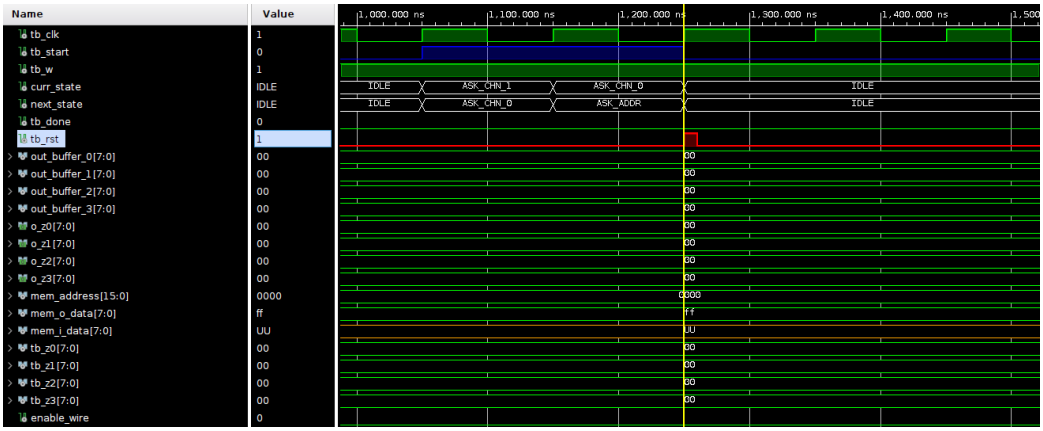
3.1.2 Test 2

Questo test bench verifica che il modulo resetti correttamente indipendentemente da quando viene alzato `i_rst`.
Nello specifico:

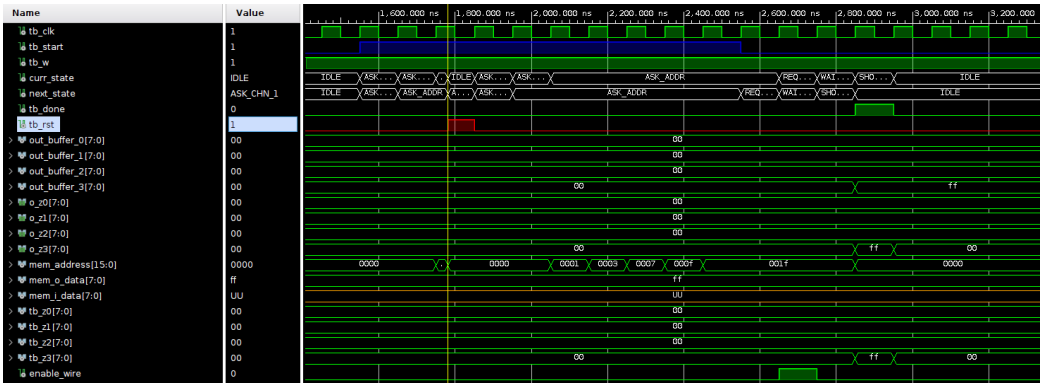
`i_rst` sale sul fronte di discesa di `o_done`



`i_rst` sale sul fronte di discesa di `i_start`



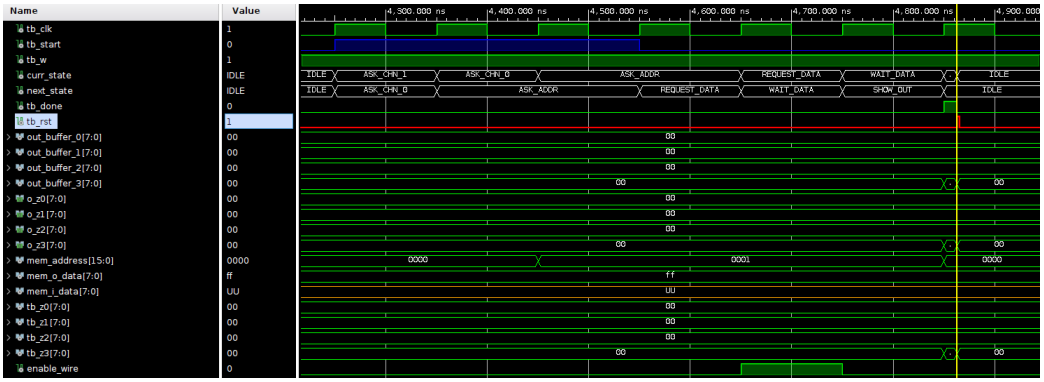
`i_rst` sale mentre `i_start` è alto (e rimane alto)



`i_rst` sale durante la computazione dell'ingresso



`i_rst` sale mentre `o_done` è alto



3.2 Sintesi

Il componente sintetizza correttamente, e supera tutti i test post-synthesis functional. I report di sintesi indicano correttamente l'utilizzo di 53 flip-flop e 0 latch. Il numero di FF utilizzati è corente con la struttura del modulo considerando:

#FF	Utilizzo
16	`shift_reg`
2	`chn`
4×8	`out_buffer_0..3`
3	7 stati del FSM

Inoltre i report indicano un ampio margine nei tempi di elaborazione:

```
Slack (MET) :     97.496ns  (required time - arrival time)
Data Path Delay:  1.990ns  (logic 0.611ns (30.704%)  route 1.379ns (69.296%))
```

4 Conclusioni

Il componente sintetizzato ha un tempo di elaborazione (da quando `i_start` torna a 0 a quando `o_done` viene posto a 1) di 3 cicli di clock, ampiamente al di sotto del limite della specifica di 20 cicli di clock. Indubbiamente sono possibili ulteriori ottimizzazioni anche a livello del numero di stati dell'FSM che tuttavia porterebbero ad un maggiore utilizzo di flip-flop dovendo introdurre un contatore ad esempio, pertanto la soluzione sintetizzata è un compromesso tra prestazioni ed utilizzo di risorse, mantenendo una discreta semplicità di implementazione.